
VersaCamVLA: Camera-Configurable VLA Policies for Robotic Manipulation

Boyao Han^{1,2*} Chen Shi^{1*} Jingjing Qian¹ Zhuotao Tian^{2,3} Li Jiang^{1,2†}

¹The Chinese University of Hong Kong, Shenzhen

²Shenzhen Loop Area Institute ³Harbin Institute of Technology, Shenzhen

Project Page: <https://boyaohan.github.io/VersaCamVLA.github.io/>

Abstract

Vision-Language-Action (VLA) models have emerged as powerful foundations for robotic manipulation, but their reliance on fixed camera configurations during training makes them brittle to changes in camera count or pose during deployment. To overcome these limitations, we propose **VersaCamVLA**, a camera-configurable framework that decouples camera-set representation from action learning. VersaCamVLA learns a unified scene-token interface that maps an arbitrary, variable set of posed RGB views into fixed-size latent scene tokens. This is achieved via multi-signal target-view prediction and Wrist-Augmented Pose Sampling (WAPS), which leverages natural wrist-camera motion for free pose diversity. At deployment, a lightweight spatial encoder injects these compact scene tokens into a pretrained base VLA as a supplementary visual condition, requiring no explicit 3D sensing or novel-view rendering. Experiments on RoboTwin, LIBERO, and a real-robot platform demonstrate that VersaCamVLA consistently outperforms prior VLA methods and direct multi-view baselines, maintaining robust performance across varying camera counts and unseen camera poses.

1 Introduction

Vision-Language-Action (VLA) models have become a powerful foundation for robotic manipulation, adapting pretrained Vision-Language Models (VLMs) to map visual observations, language instructions, and proprioception to robot actions [1–8], and their semantic priors and few-shot adaptability make them attractive as general policies across diverse tasks and environments. However, fine-grained tasks such as precise insertion, and bimanual assembly demand reliable spatial perception, which is tightly coupled to the cameras used during training: changing the number of cameras or shifting a camera’s pose alters the visual token distribution and degrades the policy. This poses a obstacle for deployment, where camera placement is often constrained by the robot, workspace, or task, and a useful view on one platform may be unavailable or suboptimal on another, as illustrated in Fig. 1.

Prior work has addressed camera sensitivity through several complementary directions. One line of work improves viewpoint robustness by collecting demonstrations from multiple camera poses [9–12] or synthesizing novel views for training-time augmentation [13] and training-free test-time adaptation [14]. However, multi-pose collection is costly on real robots, and synthesis-based methods depend on the fidelity of generated views. Another line introduces geometric priors through depth, point clouds, or rendered views [15–18], as well as geometric features extracted from RGB images by pretrained 3D foundation models [19, 20]. Finally, multi-camera policies often append visual tokens from additional cameras to the policy input [21–23]. This direct multi-view formulation improves coverage, but it still binds the policy to the camera count and view layout used during action learning, and its token length grows with the number of input views.

*: Equal Contribution. †: Corresponding author.

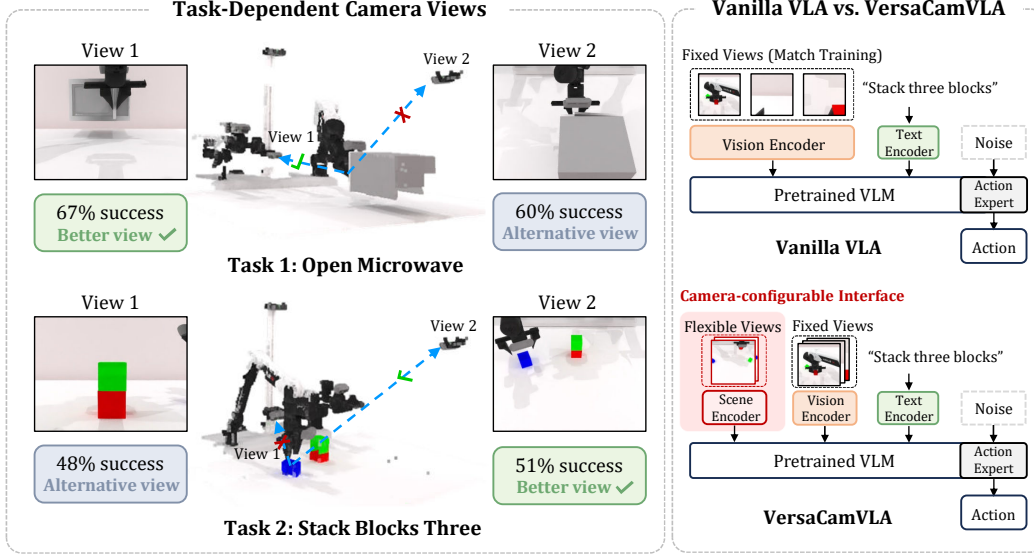


Figure 1: **Left:** Different manipulation tasks prefer different camera views, making a fixed view setup suboptimal. **Right:** VersaCamVLA augments a vanilla VLA with a camera-configurable scene-token interface that enables flexible view setup at deployment.

We argue that a VLA should instead support a camera-configurable visual pathway that accepts a variable number of posed RGB views, makes no assumption on camera poses, and presents the action policy with a unified fixed-size scene representation. To this end, we present **VersaCamVLA**, a camera-configurable VLA framework that decouples camera-set representation learning from action learning. Rather than redesigning the VLA backbone or requiring explicit 3D reconstruction, VersaCamVLA learns a unified scene-token interface between raw camera observations and the policy. This interface maps any non-empty set of posed RGB views into fixed-size scene tokens, allowing different camera configurations to be represented through the same policy-facing visual format.

VersaCamVLA is trained in two stages: unified scene-token interface learning and scene-token-conditioned VLA policy learning. In Stage I, a scene encoder aggregates a source set of posed RGB views into latent scene tokens, and a training-only scene decoder queries these tokens with target-camera rays predict RGB, semantic, and edge signals at target views. This multi-signal target-view objective encourages the scene tokens to preserve view-consistent spatial information while providing object-level and boundary-aware supervision beyond raw pixel reconstruction. To expose the encoder to diverse camera configurations without collecting additional multi-pose demonstrations, we introduce *Wrist-Augmented Pose Sampling* (WAPS), which turns the natural motion of wrist cameras into a free source of pose diversity. At each training frame, WAPS varies the source camera subset while asking the decoder to reconstruct all available views, producing both self-reconstruction and cross-view prediction signals for the same scene tokens.

In Stage II, VersaCamVLA discards the decoder and prediction heads, freezes the scene encoder, and integrates the learned scene tokens into a pretrained VLA policy. The base VLA keeps its native fixed-view RGB input and action-learning objective, while a lightweight spatial encoder compresses the learned scene tokens into a compact supplementary visual condition. Camera flexibility therefore enters through the auxiliary scene-token pathway: any available posed camera subset can be encoded into the same fixed-size representation, while the action expert receives a stable conditioning interface. At deployment, VersaCamVLA supports versatile camera setups and requires only RGB images with camera intrinsics and extrinsics. No novel-view rendering, depth estimation, point-cloud processing, or explicit 3D reconstruction is needed.

We evaluate VersaCamVLA on RoboTwin [24], LIBERO [25], and a real-robot platform with varying camera counts and poses. Across these scenarios, VersaCamVLA consistently outperforms prior works and direct multi-view baselines. These findings show that learning a unified scene-token interface effectively makes pretrained VLA policies more robust to practical camera-configuration changes. In summary, our main contributions are threefold:

- We formulate camera-configurable VLA policy learning, where a policy must operate with a variable number of posed RGB views without assuming fixed camera poses, while maintaining a fixed-size visual interface for action prediction.
- We propose VersaCamVLA, a two-stage framework that learns a unified scene-token interface through target-view prediction, then injects the resulting scene tokens into a pretrained VLA as supplementary visual conditioning. Wrist-Augmented Pose Sampling is introduced to exploit wrist-camera motion as free pose diversity for scene-token learning.
- We validate VersaCamVLA on simulation and real-robot manipulation benchmarks, showing consistent improvements over prior VLA methods and direct multi-view baselines under changing camera counts and unseen camera poses.

2 Related Work

2.1 Vision-Language-Action Models

Vision-Language-Action models (VLAs) [1–3, 22, 6, 4, 5, 26] adapt pretrained vision-language models (VLMs) to robot control by training on large-scale teleoperation and egocentric data, generalizing across novel scenes, objects, and instructions. However, recent evaluations show that VLAs are sensitive to camera-configuration shifts, especially viewpoint variations [27–29], mainly because their 2D RGB inputs lack explicit camera-pose awareness. Most VLA policies further fix the number of input cameras during post-training, limiting adaptation to camera additions, removals, or pose changes. Our work targets this underexplored camera-configuration flexibility for pretrained VLAs.

2.2 Viewpoint Generalization for Vision-Language-Action Policies

Recent work addresses viewpoint brittleness in visuomotor and Vision-Language-Action policies through data augmentation, test-time view synthesis, and geometry-aware representation learning. Data augmentation broadens viewpoint coverage through multi-pose demonstrations [9, 10, 12] or novel-view synthesis [30, 31], but is limited by collection cost and synthesis fidelity. Geometry-aware methods inject priors such as depth [17, 32], point clouds [33, 15, 34], or view-aligned 3D features [15–17, 35–37, 16], as well as geometric features extracted from RGB images by pretrained 3D foundation models [19, 20]. In contrast, our method learns compact scene tokens from posed RGB views, improving unseen-pose robustness without explicit 3D reconstruction at deployment or extra multi-pose demonstrations, while requiring calibrated camera intrinsics and extrinsics.

2.3 Multi-Camera Perception for Vision-Language-Action Policies

Multi-camera perception is a common strategy in imitation learning and robot manipulation. Prior policies [21, 22] concatenate image features or tokens from multiple cameras and feed them into a transformer-based action predictor. Recent VLA-style approaches [38, 22, 23, 39–43] adopt a similar design by appending external-view tokens to the policy input. This direct multi-view formulation is simple and effective, and has been widely used to improve scene coverage, reduce occlusion, and enhance policy performance on fine-grained manipulation tasks.

3 Methodology

We present **VersaCamVLA**, a camera-configurable VLA framework that decouples camera-set representation learning from action learning, as illustrated in Fig. 2. In Stage I, VersaCamVLA learns a unified scene-token interface between raw observations and the policy. A scene encoder maps any non-empty set of posed RGB views to fixed-size latent scene tokens, supervised by target-view prediction, which encourages the scene tokens to preserve view-consistent spatial information across camera poses. In Stage II, the decoder used for scene representation learning is removed, the scene encoder is frozen, and a compact version of the learned scene tokens is injected into a pretrained VLA policy as additional visual conditioning, allowing the policy to use observations from different numbers of cameras and unseen camera poses.

Specifically, we first introduce the camera-configurable VLA setting and base policy in Sec. 3.1. We then describe how VersaCamVLA learns the unified scene-token interface in Sec. 3.2. Sec. 3.3

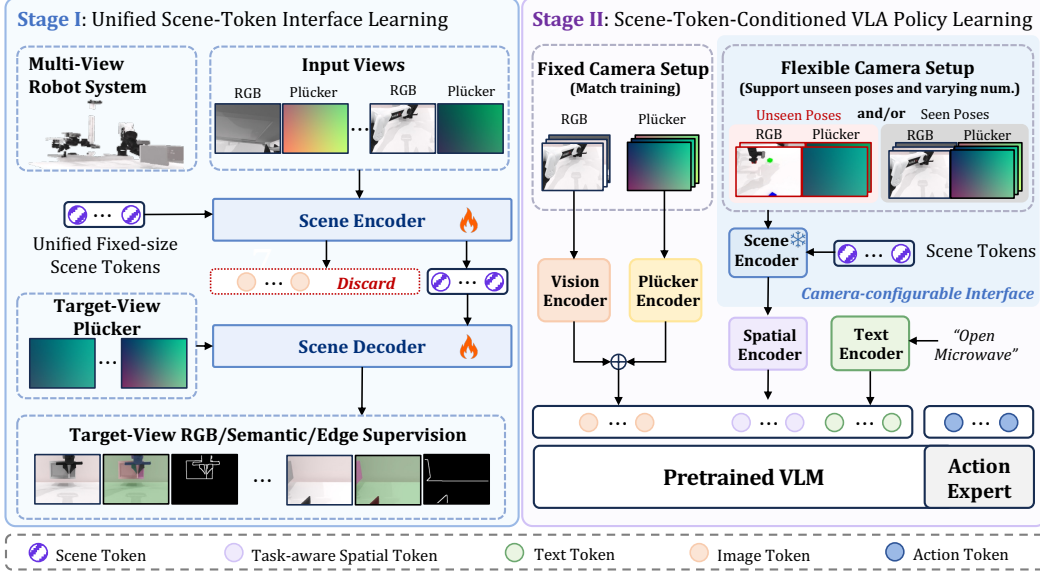


Figure 2: **Overview of VersaCamVLA.** **Stage I:** A scene encoder aggregates posed RGB views into fixed-size scene tokens, with a training-only decoder predicting RGB, semantic, and edge signals at target views. **Stage II:** The decoder is discarded, the scene encoder is frozen, and a jointly-trained spatial encoder compresses the tokens into a supplementary visual condition for a pretrained VLA.

presents Wrist-Augmented Pose Sampling, which improves camera-configuration coverage during scene representation learning. Finally, Sec. 3.4 describes how the learned scene tokens are integrated into the VLA policy for action learning.

3.1 Camera-Configurable VLA Setting

Standard VLA formulation. A VLA policy π predicts robot actions at timestep t from a language instruction $\mathcal{L}$, visual observation $\mathcal{V}_t$, and proprioceptive state $\mathcal{P}_t$, written as

$$A_t = \pi(\mathcal{L}, \mathcal{V}_t, \mathcal{P}_t) = [a_t, a_{t+1}, \dots, a_{t+H_a-1}], \quad (1)$$

where H_a is the action horizon. Each action $a_t = [\mathbf{q}_t^L, g_t^L, \mathbf{q}_t^R, g_t^R] \in \mathbb{R}^{14}$ is represented in joint space: $\mathbf{q}_t^L, \mathbf{q}_t^R \in \mathbb{R}^6$ are the left and right arm joint angles, and $g_t^L, g_t^R \in \mathbb{R}$ are the gripper states. In conventional single- or multi-view VLAs, $\mathcal{V}_t$ is tied to the camera setup used during policy training. For example, a multi-camera VLA typically instantiates $\mathcal{V}_t$ as an image tuple $\{\mathbf{I}_t^i\}_{i=1}^N$, where the number of views and the camera poses are fixed components of the policy input.

Camera-configurable observations. The fixed-tuple visual observation in standard VLAs makes the policy interface depend on the demonstrated camera setup. If a camera is added or moved to a new pose, the resulting visual tokens no longer match the structure seen during action learning. To remove this dependency, we keep the standard VLA input-output form but replace the camera component of $\mathcal{V}_t$ with a flexible camera set. Let $\mathcal{C} = \{(\mathbf{K}^i, \mathbf{T}^i)\}_{i=1}^N$ denote a camera configuration, where $\mathbf{K}^i$ and $\mathbf{T}^i$ are the intrinsic and extrinsic matrices of camera i . Under this configuration, the visual observation is $\mathcal{V}_t^{\mathcal{C}} = \{(\mathbf{I}_t^i, \mathbf{K}^i, \mathbf{T}^i)\}_{i=1}^N$. Camera-configurable VLA policy learning asks for a single policy that remains valid as $\mathcal{C}$ varies across deployment setups. The visual observation should therefore allow a variable number of views from non-fixed poses, while the downstream action policy should still receive fixed-size scene tokens. This motivates VersaCamVLA’s scene-token interface, which maps $\mathcal{V}_t^{\mathcal{C}}$ to unified scene tokens $\mathbf{Z}_t$ whose size is independent of N .

Backbone instantiation. We instantiate π with $\pi_{0.5}$ [1], a pretrained VLA that uses a VLM [44, 45] to encode visual and language context and a robotics-specific action expert to generate continuous action chunks. VersaCamVLA does not redesign the VLA backbone; instead, it augments the backbone’s visual context with scene tokens produced from $\mathcal{V}_t^{\mathcal{C}}$. The action expert follows $\pi_{0.5}$ [2] and predicts A_t through a flow-matching denoising objective, detailed in Sec. 3.4.

3.2 Unified Scene-Token Interface Learning

To implement a unified scene-token interface, VersaCamVLA uses a **scene encoder** that turns the camera-configurable observation into fixed-size scene tokens $\mathbf{Z}$. Inspired by LVSM [46], we learn this encoder with target-view prediction: a **scene decoder**, used only during representation learning, queries $\mathbf{Z}$ with target-camera rays and reconstructs RGB, semantic, and edge signals. This objective encourages $\mathbf{Z}$ to support prediction at target views beyond the input views, while presenting the downstream VLA with a consistent visual representation regardless of the input camera set.

Scene encoder. Given a source camera set $\mathcal{S}$ and the corresponding input observation $\mathcal{V}^{\mathcal{S}} = \{(\mathbf{I}^i, \mathbf{K}^i, \mathbf{T}^i)\}_{i=1}^N$ with N source views, we first compute a Plücker ray embedding map $\mathbf{R}^i$ for each posed view. We then tokenize each RGB image and ray map via ViT-style patchification with patch size p , yielding $P = H_I W_I / p^2$ patches per view. For the j -th patch of view i , we concatenate the RGB patch $\mathbf{I}^{i,j} \in \mathbb{R}^{p \times p \times 3}$ with its ray embedding patch $\mathbf{R}^{i,j} \in \mathbb{R}^{p \times p \times 6}$, and project the result into a d -dimensional source token:

$$\mathbf{x}^{i,j} = \phi_{\text{src}}([\mathbf{I}^{i,j}, \mathbf{R}^{i,j}]) \in \mathbb{R}^d, \quad (2)$$

where ϕ_{src} is a linear patch projection. The tokens from all source views are stacked into a source sequence $\mathbf{X} \in \mathbb{R}^{(N \times P) \times d}$, whose length varies with the number of input cameras.

We use L learnable scene tokens $\mathbf{E} = \{\mathbf{e}^k\}_{k=1}^L$ with $\mathbf{e}^k \in \mathbb{R}^d$ to aggregate the variable-length source-token sequence into a fixed-size representation. The source tokens and scene tokens are concatenated and processed jointly by a Transformer encoder, allowing the scene tokens to attend to spatial information from all input views through self-attention. We discard the updated source tokens and retain the updated scene-token outputs $\mathbf{Z}$ as the compact scene representation:

$$\mathbf{Z} = \text{Enc}_{\theta}([\mathbf{X}, \mathbf{E}]) \in \mathbb{R}^{L \times d}. \quad (3)$$

Since L is independent of the number and poses of input cameras, $\mathbf{Z}$ provides a fixed-size scene representation for downstream policy learning under flexible camera configurations.

Scene decoder. To encourage $\mathbf{Z}$ to encode view-consistent scene information that remains useful beyond the input camera views, we train a scene decoder to predict signals at target view poses conditioned on $\mathbf{Z}$. Given a target camera $c = (\mathbf{K}, \mathbf{T})$, we denote its tokenized Plücker ray embeddings as $\{\mathbf{R}^{c,j}\}_{j=1}^P$ and project each target ray patch through a linear projection ϕ_{tgt} :

$$\mathbf{q}^{c,j} = \phi_{\text{tgt}}(\mathbf{R}^{c,j}) \in \mathbb{R}^d. \quad (4)$$

The target ray tokens $\mathbf{Q}^c \in \mathbb{R}^{P \times d}$ are concatenated with the scene tokens and passed through a Transformer with self-attention. We keep only the updated target tokens $\mathbf{Y}^c$ as output:

$$\mathbf{Y}^c = \text{Dec}_{\psi}([\mathbf{Q}^c, \mathbf{Z}]) \in \mathbb{R}^{P \times d}. \quad (5)$$

$\mathbf{Y}^c$ are then fed to lightweight prediction heads for RGB, semantic-map, and edge-map reconstruction. Successful prediction requires $\mathbf{Z}$ to summarize the source observations in a form that supports prediction beyond the encoder input views.

Multi-signal target-view supervision. Given a target camera set $\mathcal{T}$, the scene decoder produces target-view output tokens $\mathbf{Y}^c$ for each $c \in \mathcal{T}$. A lightweight prediction head h_{rgb} then maps $\mathbf{Y}^c$ to a predicted RGB image:

$$\hat{\mathbf{I}}^c = h_{\text{rgb}}(\mathbf{Y}^c), \quad (6)$$

and is supervised against the ground-truth RGB image $\mathbf{I}^c$ by an MSE term plus a deep perceptual term $\mathcal{L}_{\text{perc}}$ computed in a pretrained feature space:

$$\mathcal{L}_{\text{rgb}} = \sum_{c \in \mathcal{T}} \left[\text{MSE}(\hat{\mathbf{I}}^c, \mathbf{I}^c) + \lambda_{\text{perc}} \mathcal{L}_{\text{perc}}(\hat{\mathbf{I}}^c, \mathbf{I}^c) \right]. \quad (7)$$

RGB reconstruction alone can be dominated by texture, lighting, and background statistics. To better align the scene tokens with manipulation-relevant structure, we add two complementary supervision signals: semantic maps and edge maps. Two additional heads on the same target-view decoder output produce the semantic and edge predictions:

$$\hat{\mathbf{S}}^c = h_{\text{sem}}(\mathbf{Y}^c), \quad \hat{\mathbf{B}}^c = h_{\text{edge}}(\mathbf{Y}^c), \quad (8)$$

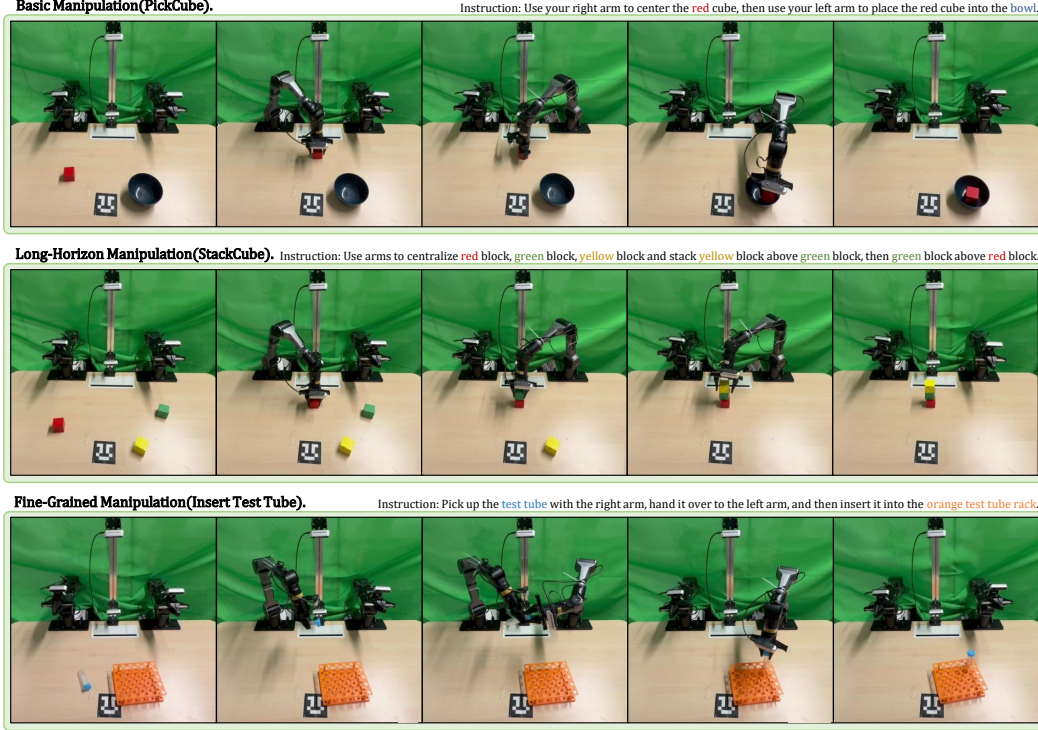


Figure 3: **Real-world Manipulation Processes.** Representative rollouts of VersaCamVLA on three real-world tasks: PickCube, StackCube, and Insert Test Tube, showing key execution states from the initial scene to task completion.

each supervised against its target map by an MSE term:

$$\mathcal{L}_{\text{sem}} = \sum_{c \in \mathcal{T}} \text{MSE}(\hat{\mathbf{S}}^c, \mathbf{S}^c), \quad \mathcal{L}_{\text{edge}} = \sum_{c \in \mathcal{T}} \text{MSE}(\hat{\mathbf{B}}^c, \mathbf{B}^c). \quad (9)$$

Semantic targets $\mathbf{S}^c$ come from the simulator’s segmentation sensor in simulation and from a SAM3 segmenter [47] fine-tuned on our real-robot data for real frames; edge targets $\mathbf{B}^c$ are extracted with a Canny detector. The scene-token learning objective is the weighted sum:

$$\mathcal{L}_{\text{scene}} = \mathcal{L}_{\text{rgb}} + \lambda_{\text{sem}} \mathcal{L}_{\text{sem}} + \lambda_{\text{edge}} \mathcal{L}_{\text{edge}}, \quad (10)$$

where λ_{sem} and λ_{edge} balance the high-level signals against photometric reconstruction.

3.3 Wrist-Augmented Pose Sampling

The scene encoder in Sec. 3.2 is designed to accept a variable number of views from non-fixed poses and produce fixed-size unified scene tokens for downstream VLA. Learning this camera-configurable interface, however, requires the training data to cover sufficiently diverse camera placements. In tabletop manipulation data, static cameras provide stable but sparsely distributed viewpoints, while wrist cameras sweep a much richer pose distribution as the robot moves. Wrist-Augmented Pose Sampling (WAPS) turns this natural motion into pose diversity for scene-token learning.

Specifically, for each training frame, let $\mathcal{C}_{\text{stat}}$ and $\mathcal{C}_{\text{wrist}}$ denote the available static-camera and wrist-camera views. The source view set $\mathcal{S} = \mathcal{S}_{\text{stat}} \cup \mathcal{S}_{\text{wrist}}$ fed to the scene encoder is drawn from the two camera types independently. From the static cameras, we randomly sample n views as $\mathcal{S}_{\text{stat}}$, where n is a random integer between 1 and $|\mathcal{C}_{\text{stat}}|$. For wrist cameras, each view is independently included in $\mathcal{S}_{\text{wrist}}$ with probability 0.5. We use the full camera pool as the target set $\mathcal{T} = \mathcal{C}_{\text{stat}} \cup \mathcal{C}_{\text{wrist}}$, so the decoder is always trained to render every available view, regardless of the encoder input.

This design provides two complementary supervision signals on the same scene tokens. Self-reconstruction on source views preserves visual detail through the latent bottleneck, while cross-view prediction on held-out views forces $\mathbf{Z}$ to support view poses that were not in the encoder input.

Table 1: **Simulation results on RoboTwin 2.0.** We report success rates (%) on RoboTwin 2.0. All results are reproduced by us. **Bold** indicates the best performance in each column.

Task	ACT[21]		DP[48]		DP3[15]		OpenVLA-OFT[3]		$\pi_{0.5}$ [1]		VersaCamVLA	
	Clean	DR	Clean	DR	Clean	DR	Clean	DR	Clean	DR	Clean	DR
Blocks Ranking RGB	0	0	0	0	2	0	0	0	38	11	67	12
Blocks Ranking Size	3	0	1	0	2	1	7	0	12	4	35	6
Hanging Mug	6	0	19	0	35	1	12	0	4	4	10	4
Move Stapler Pad	0	0	0	0	7	0	0	0	5	7	8	5
Open Laptop	72	1	53	0	77	1	82	0	90	70	96	62
Open Microwave	72	0	78	0	91	25	23	56	43	26	51	46
Pick Diverse Bottles	8	0	28	0	55	1	9	0	36	21	47	12
Place A2B Left	1	0	3	0	30	1	5	0	42	24	54	17
Place A2B Right	2	0	6	0	43	0	11	0	24	19	45	27
Place Burger Fries	67	0	80	0	75	2	30	0	48	31	81	49
Place Dual Shoes	0	0	4	0	9	0	2	0	25	24	51	25
Place Empty Cup	38	0	27	0	75	1	20	0	77	1	81	27
Place Object Basket	1	0	21	0	49	3	5	0	53	26	76	35
Scan Object	3	0	8	0	29	1	8	1	6	4	35	15
Stack Blocks Three	5	0	0	0	3	0	0	0	18	50	43	3
Stack Bowls Three	37	0	51	0	57	4	29	0	53	24	61	23
Average (%)	19.69	0.06	23.69	0.00	39.94	2.56	15.19	3.56	35.88	21.63	52.56	23.00

Table 2: **Simulation results on LIBERO.** We report task success rates (%) on LIBERO. Results marked with * are reproduced by us. **Bold** indicates the best performance in each column, and underlined indicates the second-best performance.

Method	LIBERO-Spatial	LIBERO-Goal	LIBERO-Object	LIBERO-Long	Average
Diffusion Policy[48]	78.3	92.5	68.3	50.5	72.4
OpenVLA-OFT [3]	95.2	94.2	95.2	93.2	94.5
$\pi_{0.5}^*$ [1]	98.0	94.6	99.0	91.6	<u>95.8</u>
VersaCamVLA	98.2	95.0	<u>97.8</u>	<u>92.4</u>	95.9

By varying the number and poses of source cameras across training frames, the encoder learns to extract scene representation from arbitrary camera subsets instead of relying on a fixed camera setup. Wrist-camera motion further provides diverse cross-view targets at no extra data-collection cost.

3.4 Scene-Token-Conditioned VLA Policy Learning

After learning the scene-token interface, we discard the scene decoder and prediction heads, freeze the scene encoder, and integrate the resulting scene tokens into the base VLA for action learning.

Action policy learning. At time step t , the frozen scene encoder maps the versatile camera configuration $\mathcal{C}$ and the corresponding posed views $\mathcal{V}_t^C$ into scene tokens $\mathbf{Z}_t$ following Sec. 3.2. Although $\mathbf{Z}_t$ has fixed length, directly feeding all scene tokens into the VLA can still add unnecessary computation and may expose the policy to background details that are irrelevant to action prediction. A convolutional spatial encoder g_η compresses scene tokens into a compact, task-oriented representation:

$$\bar{\mathbf{Z}}_t = g_\eta(\mathbf{Z}_t). \quad (11)$$

The compact $\bar{\mathbf{Z}}_t$ is fed into VLA as a supplementary scene representation. The policy therefore conditions on the language instruction $\mathcal{L}$, the RGB observations $\mathcal{V}_t$, the supplementary scene representation $\bar{\mathbf{Z}}_t$, and the proprioceptive state $\mathcal{P}_t$, and predicts the action chunk A_t .

The scene encoder stays frozen throughout this stage; g_η is randomly initialized and, jointly with base VLA’s trainable parameters ω , optimized by the standard flow-matching action regression loss:

$$\mathcal{L}_{\text{action}} = \mathbb{E}_{\tau, \epsilon} \left[\|v_{\eta, \omega}(A_t^\tau; \mathcal{L}, \mathcal{V}_t, \bar{\mathbf{Z}}_t, \mathcal{P}_t, \tau) - (A_t - \epsilon)\|_2^2 \right], \quad (12)$$

where $\epsilon \sim \mathcal{N}(0, \mathbf{I})$, $\tau \sim \mathcal{U}[0, 1]$, $A_t^\tau = \tau A_t + (1 - \tau)\epsilon$ is the linear interpolation between noise and the demonstration action chunk, and $v_{\eta, \omega}$ is the velocity field predicted by the base VLA $\pi_{0.5}$ ’s action expert. Because g_η is supervised only through $\mathcal{L}_{\text{action}}$, it acts as a task-driven information bottleneck that retains scene-token information useful for action prediction.

Table 3: **Robustness to camera-pose variations.** Average success rate (%) on RoboTwin 2.0 *Clean* and real-robot tasks. **Bold** indicates the best performance in each column.

Method	RoboTwin 2.0		Real Robot	
	Seen Pose	Unseen Pose	Seen Pose	Unseen Pose
$\pi_{0.5}$	48.88	32.88	41.67	13.33
VersaCamVLA	53.44	52.63	53.33	58.33

Table 4: **Effect of input camera count.** Average success rate (%) on RoboTwin 2.0 *Clean* and real-robot tasks. V denotes views. **Bold** indicates the best performance in each column.

Method	RoboTwin 2.0				Real Robot		
	3V	4V	5V	6V	3V	4V	5V
$\pi_{0.5}$	35.88	48.88	x	x	31.67	41.67	x
VersaCamVLA	52.56	53.44	53.13	55.63	x	53.33	65.00

Inference. At deployment, the scene decoder and the three prediction heads are discarded; only the scene encoder, spatial encoder, and the base VLA backbone ($\pi_{0.5}$) run on the inference path. Any subset of posed camera observations available on the robot system is encoded into the same fixed-length $\mathbf{Z}_t$, so the policy operates under flexible camera configurations through a unified scene-token interface, without novel-view rendering, depth, point clouds, or explicit 3D reconstruction.

4 Experiments

4.1 Experimental Setup

Simulation Setup. We evaluate on two standard robotic manipulation benchmarks, **RoboTwin 2.0** [24] for bimanual manipulation under canonical (*Clean*) and domain-randomized (*DR*) settings, and **LIBERO** [25] for single-arm manipulation across four suites. Embodiments, task selection, demonstration counts, and training protocols are described in Appendix ??.

Camera-Configuration Setup. Beyond the official benchmark settings, we design additional evaluations to test robustness to changing camera configurations.

- **Camera-Pose Perturbation.** On both RoboTwin 2.0 and LIBERO, policies are trained under the nominal camera setup and evaluated with camera poses perturbed in azimuth, distance, and pitch over a workspace-centered spherical region, probing generalization to unseen viewpoints.
- **Camera-Count Variation.** On RoboTwin 2.0 *Clean*, we train two $\pi_{0.5}$ baselines separately at 3 and 4 views and a single VersaCamVLA policy, then evaluate VersaCamVLA under 3, 4, 5, and 6 input views and each $\pi_{0.5}$ baseline at its matched view count, isolating the effect of camera count.

Real-World Setup. To evaluate real-robot camera flexibility, we test VersaCamVLA on three Cobot Magic bimanual tasks, covering sequential coordination, long-horizon manipulation, and fine handover-and-insertion, with 20 trajectories per task. Figure 3 provides visualizations of all tasks.

- **Basic Manipulation (PickCube).** The right arm moves a red cube to the workspace center, and the left arm then picks it up and places it into a bowl, exercising sequential bimanual coordination in a shared workspace.
- **Long-Horizon Manipulation (StackCube).** The robot stacks three cubes into a red–green–yellow tower from bottom to top with cube positions randomized at evaluation; small placement errors collapse the tower, stressing precise spatial control over an extended horizon.
- **Fine-Grained Manipulation (Insert Test Tube).** The right arm grasps a test tube, transfers it to the left arm, and inserts it into a tube rack, requiring accurate handover orientation and fine local alignment despite gripper-induced occlusions during the handover.

Baselines. We compare VersaCamVLA with visuomotor and bimanual VLA baselines. Visuomotor policies include ACT [21], DP [48] and DP3 [15] trained separately per task. VLA baselines include OpenVLA-OFT [3], the original $\pi_{0.5}$ [1]. For $\pi_{0.5}$, each additional RGB view is encoded with SigLIP [49] and appended to the policy input as extra visual tokens in a fixed view order.

Table 5: **Token ablation on RoboTwin 2.0 Clean.** Average success rate (%). **Bold** indicates the best performance.

Token Number	48	192	432
Average SR (%)	50.06	55.63	47.50

Table 7: **WAPS ablation.** Average success rate (%) under the unseen camera pose setting. **Bold** indicates the best result.

Method	w/o WAPS	WAPS
Average SR (%)	48.79	52.63

Table 6: **Spatial Encoder ablation on RoboTwin 2.0 Clean.** Average success rate (%). **Bold** indicates the best performance.

Spatial Encoder	Pooling	Conv
Average SR (%)	50.13	55.63

Table 8: **LVSM pretraining ablation.** Average success rate (%) on RoboTwin 2.0 Clean. **Bold** indicates the best result.

Initialization	From scratch	Pretrained LVSM
Average SR (%)	51.25	55.63

Implementation Details. We use a predictive horizon of $H = 50$ for all tasks. Details of the camera-pose perturbation sampling protocol are provided in Appendix ???. We train the policy for 30,000 iterations using AdamW [50] with a learning rate of 5×10^{-5} on 8 NVIDIA H100 GPUs and a total batch size of 512. The primary evaluation metric is task success rate (%). More implementation details are provided in Appendix ??.

4.2 Main Results

Simulation Results.

- **RoboTwin 2.0.** Table 1 reports per-task success rates on the 16 fine-grained RoboTwin 2.0 tasks under the official *Clean* and DR settings, with 100 trials per task. Both $\pi_{0.5}$ and VersaCamVLA use 3-view inputs. VersaCamVLA achieves the best average in both settings, reaching 52.56% on *Clean* and 23.00% on DR. VersaCamVLA achieves the best DR average, showing that scene tokens provide robust policy representations under appearance randomization.
- **LIBERO.** Table 2 reports results on the four LIBERO suites, testing whether the scene-token pathway preserves the general manipulation capability of the pretrained backbone on standard single-arm tasks. VersaCamVLA reaches an average success rate of 95.9%, substantially above Diffusion Policy[48] (72.4%) and OpenVLA-OFT[3] (94.5%) and on par with the strong $\pi_{0.5}$ [1] baseline (95.8%), with slight gains on LIBERO-Spatial, LIBERO-Goal, and LIBERO-Long.

Camera-Configuration Results.

- **Camera-Pose Perturbation.** Table 3 summarizes camera-pose robustness on both RoboTwin 2.0 and real-robot tasks. The LIBERO counterpart deferred to the Appendix ???. Both $\pi_{0.5}$ and VersaCamVLA are evaluated under matched four-view inputs, and under perturbation $\pi_{0.5}$ drops 16.0 pp while VersaCamVLA loses only 0.8 pp. These results show that the scene-token interface is more pose-robust than direct multi-view token concatenation.
- **Camera-Count Variation.** Table 4 summarizes the effect of camera-count on both RoboTwin 2.0 and real-robot tasks. On RoboTwin 2.0 *Clean*, we train two $\pi_{0.5}$ baselines separately at 3 and 4 views and a single VersaCamVLA policy, then evaluate VersaCamVLA under 3, 4, 5, and 6 input views and each $\pi_{0.5}$ baseline at its matched view count, isolating the effect of camera count.

Real-world Results. Tables 3 and 4 report real-robot success rates on three bimanual manipulation tasks; the same VersaCamVLA model is evaluated at the trained pose, at an unseen pose, and with one extra test-time camera, while $\pi_{0.5}$ is trained separately at 3 and 4 views and tested at the matched count. Under matched 4-view inputs, VersaCamVLA outperforms $\pi_{0.5}$ by 11.7 and 45.0 percentage points on seen and unseen camera poses, respectively.

4.3 Ablation Study

We ablate five design choices in VersaCamVLA, covering how the scene-token interface is learned and how it conditions the action policy.

Number of compressed scene tokens. Table 5 varies the number of compressed scene tokens passed to the policy. Using 192 tokens achieves the highest average success of 55.63%, compared with 50.06% for 48 tokens and 47.50% for 432 tokens.

Table 9: **Ablation of supervision signals for scene-token learning.** Average success rate (%) on RoboTwin 2.0 DR. **Bold** indicates the best result.

RGB	✓	✓	✓	✓
Edge	✗	✓	✗	✓
Semantic	✗	✗	✓	✓
Average SR	20.94	24.75	25.69	29.50

Spatial encoder architecture. The spatial encoder compresses scene tokens into a supplementary visual condition for the action policy. As shown in Table 6, convolutional compression achieves 55.63% average success, compared with 50.13% for pooling.

Wrist-Augmented Pose Sampling. Learning a camera-configurable interface requires exposure to diverse camera poses. Table 7 shows that WAPS improves average success under unseen camera poses from 48.79% to 52.63%, a gain of 3.84 percentage points.

Scene encoder–decoder initialization. Table 8 examines the contribution of LVSM initialization. Initializing the scene encoder–decoder from a pretrained LVSM checkpoint achieves 55.63% average success, compared with 51.25% when trained from scratch.

Multi-signal supervision. RGB reconstruction provides a basic learning signal, while semantic and edge targets introduce object-level and boundary information. Table 9 shows that adding edge or semantic supervision individually improves average success from 20.94% to 24.75% and 25.69%, respectively. Combining both signals yields the best result of 29.50%, an improvement of 8.56 percentage points over RGB-only supervision.

5 Conclusion

We present **VersaCamVLA**, a camera-configurable VLA framework that augments pretrained policies with a unified scene-token interface decoupling camera-set representation from action learning. VersaCamVLA introduces a scene encoder learned by multi-signal target-view prediction and *Wrist-Augmented Pose Sampling* that exploits wrist-camera motion as free pose diversity, producing compact scene tokens that condition a pretrained VLA without explicit 3D sensing. Experiments on RoboTwin 2.0, LIBERO, and a real-robot platform show consistent improvements over prior VLAs and direct multi-view baselines under camera-pose perturbations and varying camera counts.

Limitations and broader impacts. VersaCamVLA assumes calibrated camera intrinsics and extrinsics and focuses on tabletop bimanual setups with manual camera placement, lacking autonomous viewpoint selection. Extending the scene-token interface to uncalibrated cameras, active perception that optimizes camera configurations online, and mobile workspaces remains key future work. In terms of impact, we hope camera-configurable VLA policies lower the deployment barrier across robot platforms by reducing reliance on a fixed camera setup. However, safety-critical deployment requires further validation against task-specific failures under large camera-configuration shifts.

References

- [1] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- [2] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [3] Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success, 2025. URL <https://arxiv.org/abs/2502.19645>.
- [4] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Tomas Jackson, Sally Jesmonth, Nikhil Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Isabel Leal, Kuang-Huei Lee, Sergey Levine, Yao Lu, Utsav Malla, Deeksha Manjunath, Igor Mordatch, Ofir Nachum, Carolina Parada, Jodilyn Peralta, Emily Perez, Karl Pertsch, Jornell Quiambao, Kanishka Rao, Michael

- Ryoo, Grecia Salazar, Pannag Sanketi, Kevin Sayed, Jaspiar Singh, Sumedh Sontakke, Austin Stone, Clayton Tan, Huong Tran, Vincent Vanhoucke, Steve Vega, Quan Vuong, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Tianhe Yu, and Brianna Zitkovich. Rt-1: Robotics transformer for real-world control at scale. In *arXiv preprint arXiv:2212.06817*, 2022.
- [5] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183. PMLR, 2023.
- [6] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Charles Xu, Jianlan Luo, Tobias Kreiman, You Liang Tan, Lawrence Yunliang Chen, Pannag Sanketi, Quan Vuong, Ted Xiao, Dorsa Sadigh, Chelsea Finn, and Sergey Levine. Octo: An open-source generalist robot policy. In *Proceedings of Robotics: Science and Systems*, Delft, Netherlands, 2024.
- [7] NVIDIA, :, Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi "Jim" Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, Joel Jang, Zhenyu Jiang, Jan Kautz, Kaushil Kundalia, Lawrence Lao, Zhiqi Li, Zongyu Lin, Kevin Lin, Guilin Liu, Edith Llonet, Loic Magne, Ajay Mandlekar, Avnish Narayan, Soroush Nasiriany, Scott Reed, You Liang Tan, Guanzhi Wang, Zu Wang, Jing Wang, Qi Wang, Jiannan Xiang, Yuqi Xie, Yinzhen Xu, Zhenjia Xu, Seonghyeon Ye, Zhiding Yu, Ao Zhang, Hao Zhang, Yizhou Zhao, Ruijie Zheng, and Yuke Zhu. Gr00t n1: An open foundation model for generalist humanoid robots, 2025. URL <https://arxiv.org/abs/2503.14734>.
- [8] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan P Foster, Pannag R Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. Openvla: An open-source vision-language-action model. In Pulkit Agrawal, Oliver Kroemer, and Wolfram Burgard, editors, *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 2679–2713. PMLR, 06–09 Nov 2025. URL <https://proceedings.mlr.press/v270/kim25c.html>.
- [9] Tianchong Jiang, Jingtian Ji, Xiangshan Tan, Jiading Fang, Anand Bhattad, Vitor Guizilini, and Matthew R Walter. Do you know where your camera is? view-invariant policy learning with camera conditioning. *arXiv preprint arXiv:2510.02268*, 2025.
- [10] Jinyue Bian, Zhaoxing Zhang, Zhengyu Liang, Shiwei Zheng, Shengtao Zhang, Rong Shen, Chen Yang, and Anzhou Hou. Vla-lpaf: Lightweight perspective-adaptive fusion for vision-language-action to enable more unconstrained robotic manipulation. *arXiv preprint arXiv:2509.18183*, 2025.
- [11] Alexander Khazatsky, Karl Pertsch, Suraj Nair, Ashwin Balakrishna, Sudeep Dasari, Siddharth Karamcheti, Soroush Nasiriany, Mohan Kumar Srirama, Lawrence Yunliang Chen, Kirsty Ellis, Peter David Fagan, Joey Hejna, Masha Itkina, Marion Lepert, Yecheng Jason Ma, Patrick Tree Miller, Jimmy Wu, Suneel Belkhal, Shivin Dass, Huy Ha, Arhan Jain, Abraham Lee, Youngwoon Lee, Marius Memmel, Sungjae Park, Ilija Radosavovic, Kaiyuan Wang, Albert Zhan, Kevin Black, Cheng Chi, Kyle Beltran Hatch, Shan Lin, Jingpei Lu, Jean Mercat, Abdul Rehman, Pannag R Sanketi, Archit Sharma, Cody Simpson, Quan Vuong, Homer Rich Walke, Blake Wulfe, Ted Xiao, Jonathan Heewon Yang, Arefeh Yavary, Tony Z. Zhao, Christopher Agia, Rohan Baijal, Mateo Guaman Castro, Daphne Chen, Qiuyu Chen, Trinity Chung, Jaimyn Drake, Ethan Paul Foster, Jensen Gao, Vitor Guizilini, David Antonio Herrera, Minh Heo, Kyle Hsu, Jiaheng Hu, Muhammad Zubair Irshad, Donovan Jackson, Charlotte Le, Yunshuang Li, Kevin Lin, Roy Lin, Zehan Ma, Abhiram Maddukuri, Suvir Mirchandani, Daniel Morton, Tony Nguyen, Abigail O’Neill, Rosario Scalise, Derick Seale, Victor Son, Stephen Tian, Emi Tran, Andrew E. Wang, Yilin Wu, Annie Xie, Jingyun Yang, Patrick Yin, Yunchu Zhang, Osbert Bastani, Glen Berseth, Jeannette Bohg, Ken Goldberg, Abhinav Gupta, Abhishek Gupta, Dinesh Jayaraman, Joseph J Lim, Jitendra Malik, Roberto Martín-Martín, Subramanian Ramamoorthy, Dorsa Sadigh, Shuran Song, Jiajun Wu, Michael C. Yip, Yuke Zhu, Thomas Kollar, Sergey Levine, and Chelsea Finn. Droid: A large-scale in-the-wild robot manipulation dataset, 2025. URL <https://arxiv.org/abs/2403.12945>.
- [12] Yichen Xie, Yixiao Wang, Shuqi Zhao, Cheng-En Wu, Masayoshi Tomizuka, Jianwen Xie, and Hao-Shu Fang. Multi-camera view scaling for data-efficient robot imitation learning, 2026. URL <https://arxiv.org/abs/2604.00557>.
- [13] Allan Zhou, Moo Jin Kim, Lirui Wang, Pete Florence, and Chelsea Finn. Nerf in the palm of your hand: Corrective augmentation for robotics via novel-view synthesis, 2023. URL <https://arxiv.org/abs/2301.08556>.
- [14] Hyeonjun Heo, Seungyeon Woo, Sang Min Kim, Junho Kim, Junho Lee, Yonghyeon Lee, and Young Min Kim. Anycamvla: Zero-shot camera adaptation for viewpoint robust vision-language-action models, 2026. URL <https://arxiv.org/abs/2603.05868>.

- [15] Yanjie Ze, Gu Zhang, Kangning Zhang, Chenyuan Hu, Muhan Wang, and Huazhe Xu. 3d diffusion policy: Generalizable visuomotor policy learning via simple 3d representations, 2024. URL <https://arxiv.org/abs/2403.03954>.
- [16] Zhenyang Liu, Yongchong Gu, Yikai Wang, Xiangyang Xue, and Yanwei Fu. Activevla: Injecting active perception into vision-language-action models for precise 3d robotic manipulation, 2026. URL <https://arxiv.org/abs/2601.08325>.
- [17] Jingjing Qian, Boyao Han, Chen Shi, Lei Xiao, Long Yang, Shaoshuai Shi, and Li Jiang. Geopredict: Leveraging predictive kinematics and 3d gaussian geometry for precise vla manipulation. *arXiv preprint arXiv:2512.16811*, 2025.
- [18] Albert Wilcox, Mohamed Ghanem, Masoud Moghani, Pierre Barroso, Benjamin Joffe, and Animesh Garg. Adapt3r: Adaptive 3d scene representation for domain transfer in imitation learning, 2025. URL <https://arxiv.org/abs/2503.04877>.
- [19] Ali Abouzeid, Malak Mansour, Qinbo Sun, Zezhou Sun, and Dezhen Song. Geoaware-vla: Implicit geometry aware vision-language-action model, 2026. URL <https://arxiv.org/abs/2509.14117>.
- [20] Fuhao Li, Wenxuan Song, Han Zhao, Jingbo Wang, Pengxiang Ding, Donglin Wang, Long Zeng, and Haoang Li. Spatial forcing: Implicit spatial representation alignment for vision-language-action model, 2025. URL <https://arxiv.org/abs/2510.12276>.
- [21] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware, 2023. URL <https://arxiv.org/abs/2304.13705>.
- [22] Xuewu Lin, Tianwei Lin, Lichao Huang, Hongyu Xie, Yiwei Jin, Keyu Li, and Zhizhong Su. Sem: Enhancing spatial understanding for robust robot manipulation, 2025. URL <https://arxiv.org/abs/2505.16196>.
- [23] Qingyu Fan, Zhaoxiang Li, Yi Lu, Wang Chen, Qiu Shen, Xiao xiao Long, Yinghao Cai, Tao Lu, Shuo Wang, and Xun Cao. Peafowl: Perception-enhanced multi-view vision-language-action for bimanual manipulation, 2026. URL <https://arxiv.org/abs/2601.17885>.
- [24] Tianxing Chen, Zanzin Chen, Baijun Chen, Zijian Cai, Yibin Liu, Zixuan Li, Qiwei Liang, Xianliang Lin, Yiheng Ge, Zhenyu Gu, et al. Robotwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. *arXiv preprint arXiv:2506.18088*, 2025.
- [25] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023.
- [26] Qixiu Li, Yaobo Liang, Zeyu Wang, Lin Luo, Xi Chen, Mozheng Liao, Fangyun Wei, Yu Deng, Sicheng Xu, Yizhong Zhang, Xiaofan Wang, Bei Liu, Jianlong Fu, Jianmin Bao, Dong Chen, Yuanchun Shi, Jiaolong Yang, and Baining Guo. Cogact: A foundational vision-language-action model for synergizing cognition and action in robotic manipulation, 2024. URL <https://arxiv.org/abs/2411.19650>.
- [27] Senyu Fei, Siyin Wang, Junhao Shi, Zihao Dai, Jikun Cai, Pengfang Qian, Li Ji, Xinzhe He, Shiduo Zhang, Zhaoye Fei, et al. Libero-plus: In-depth robustness analysis of vision-language-action models. *arXiv preprint arXiv:2510.13626*, 2025.
- [28] Jensen Gao, Suneel Belkhale, Sudeep Dasari, Ashwin Balakrishna, Dhruv Shah, and Dorsa Sadigh. A taxonomy for evaluating generalist robot manipulation policies. *IEEE Robotics and Automation Letters*, 2026.
- [29] Zhijie Wang, Zhehua Zhou, Jiayang Song, Yuheng Huang, Zhan Shu, and Lei Ma. Vlatest: Testing and evaluating vision-language-action models for robotic manipulation. *Proceedings of the ACM on Software Engineering*, 2(FSE):1615–1638, 2025. ISSN 2994-970X. doi: 10.1145/3729343. URL <http://dx.doi.org/10.1145/3729343>.
- [30] Stephen Tian, Blake Wulfe, Kyle Sargent, Katherine Liu, Sergey Zakharov, Vitor Guizilini, and Jiajun Wu. View-invariant policy learning via zero-shot novel view synthesis, 2025. URL <https://arxiv.org/abs/2409.03685>.
- [31] Lawrence Yunliang Chen, Chenfeng Xu, Karthik Dharmarajan, Muhammad Zubair Irshad, Richard Cheng, Kurt Keutzer, Masayoshi Tomizuka, Quan Vuong, and Ken Goldberg. Rovi-aug: Robot and viewpoint augmentation for cross-embodiment robot learning. In *Conference on Robot Learning (CoRL)*, Munich, Germany, 2024.

- [32] Jiahui Zhang, Yurui Chen, Yueming Xu, Ze Huang, Yanpeng Zhou, Yu-Jie Yuan, Xinyue Cai, Guowei Huang, Xingyue Quan, Hang Xu, and Li Zhang. 4d-vla: Spatiotemporal vision-language-action pretraining with cross-scene calibration, 2025. URL <https://arxiv.org/abs/2506.22242>.
- [33] Wenlong Huang, Yu-Wei Chao, Arsalan Mousavian, Ming-Yu Liu, Dieter Fox, Kaichun Mo, and Li Fei-Fei. Pointworld: Scaling 3d world models for in-the-wild robotic manipulation, 2026. URL <https://arxiv.org/abs/2601.03782>.
- [34] Skand Peri, Iain Lee, Chanh Kim, Li Fuxin, Tucker Hermans, and Stefan Lee. Point cloud models improve visual robustness in robotic learners, 2024. URL <https://arxiv.org/abs/2404.18926>.
- [35] Delin Qu, Haoming Song, Qizhi Chen, Yuanqi Yao, Xinyi Ye, Yan Ding, Zhigang Wang, JiaYuan Gu, Bin Zhao, Dong Wang, et al. Spatialvla: Exploring spatial representations for visual-language-action model. *arXiv preprint arXiv:2501.15830*, 2025.
- [36] Guanxing Lu, Baoxiong Jia, Puhao Li, Yixin Chen, Ziwei Wang, Yansong Tang, and Siyuan Huang. Gwm: Towards scalable gaussian world models for robotic manipulation. *Proceedings of International Conference on Computer Vision (ICCV)*, 2025.
- [37] Wenyao Zhang, Hongsi Liu, Zekun Qi, Yunan Wang, Xinqiang Yu, Jiazha Zhang, Runpei Dong, Jiawei He, He Wang, Zhizheng Zhang, Li Yi, Wenjun Zeng, and Xin Jin. Dreamvla: A vision-language-action model dreamed with comprehensive world knowledge. *CoRR*, abs/2507.04447, 2025. doi: 10.48550/ARXIV.2507.04447. URL <https://doi.org/10.48550/arXiv.2507.04447>.
- [38] Ankit Goyal, Jie Xu, Yijie Guo, Valts Blukis, Yu-Wei Chao, and Dieter Fox. Rvt: Robotic view transformer for 3d object manipulation. In Jie Tan, Marc Toussaint, and Kourosh Darvish, editors, *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 694–710. PMLR, 06–09 Nov 2023. URL <https://proceedings.mlr.press/v229/goyal23a.html>.
- [39] Haosheng Li, Weixin Mao, Zihan Lan, Hongwei Xiong, Hongan Wang, Chenyang Si, Ziwei Liu, Xiaoming Deng, and Hua Chen. Bfa++: Hierarchical best-feature-aware token prune for multi-view vision language action model, 2026. URL <https://arxiv.org/abs/2602.20566>.
- [40] Jing-Cheng Pang, Nan Tang, kaiyuan Li, Yuting Tang, Xin-Qiang Cai, Zhen-Yu Zhang, Gang Niu, Sugiyama Masashi, and Yang Yu. Learning view-invariant world models for visual robotic manipulation. In *International Conference on Learning Representations (ICLR)*, 2025.
- [41] Shengyi Qian, Kaichun Mo, Valts Blukis, David F. Fouhey, Dieter Fox, and Ankit Goyal. 3d-mvp: 3d multiview pretraining for manipulation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 22530–22539, June 2025.
- [42] Theophile Gervet, Zhou Xian, Nikolaos Gkanatsios, and Katerina Fragkiadaki. Act3d: 3d feature field transformers for multi-task robotic manipulation, 2023. URL <https://arxiv.org/abs/2306.17817>.
- [43] Pierre-Louis Guhur, Shizhe Chen, Ricardo Garcia, Makarand Tapaswi, Ivan Laptev, and Cordelia Schmid. Instruction-driven history-aware policies for robotic manipulations, 2022. URL <https://arxiv.org/abs/2209.04899>.
- [44] Lucas Beyer, Andreas Steiner, André Susano Pinto, Alexander Kolesnikov, Xiao Wang, Daniel Salz, Maxim Neumann, Ibrahim Alabdulmohsin, Michael Tschannen, Emanuele Bugliarello, Thomas Unterthiner, Daniel Keysers, Skanda Koppula, Fangyu Liu, Adam Grycner, Alexey Gritsenko, Neil Houlsby, Manoj Kumar, Keran Rong, Julian Eisenschlos, Rishabh Kabra, Matthias Bauer, Matko Bošnjak, Xi Chen, Matthias Minderer, Paul Voigtlaender, Ioana Bica, Ivana Balazevic, Joan Puigcerver, Pinelopi Papalampidi, Olivier Henaff, Xi Xiong, Radu Soricut, Jeremiah Harmsen, and Xiaohua Zhai. Paligemma: A versatile 3b vlm for transfer, 2024. URL <https://arxiv.org/abs/2407.07726>.
- [45] Jinze Bai, Shuai Bai, Shusheng Yang, Shijie Wang, Sinan Tan, Peng Wang, Junyang Lin, Chang Zhou, and Jingren Zhou. Qwen-vl: A versatile vision-language model for understanding, localization, text reading, and beyond, 2023. URL <https://arxiv.org/abs/2308.12966>.
- [46] Haian Jin, Hanwen Jiang, Hao Tan, Kai Zhang, Sai Bi, Tianyuan Zhang, Fujun Luan, Noah Snavely, and Zexiang Xu. Lvsm: A large view synthesis model with minimal 3d inductive bias. *arXiv preprint arXiv:2410.17242*, 2024.
- [47] Nicolas Carion, Laura Gustafson, Yuan-Ting Hu, Shoubhik Debnath, Ronghang Hu, Didac Suris, Chaitanya Ryali, Kalyan Vasudev Alwala, Haitham Khedr, Andrew Huang, Jie Lei, Tengyu Ma, Baishan Guo, Arpit Kalla, Markus Marks, Joseph Greer, Meng Wang, Peize Sun, Roman Rädle, Triantafyllos Afouras, Effrosyni Mavroudi, Katherine Xu, Tsung-Han Wu, Yu Zhou, Liliane Momeni, Rishi Hazra, Shuangrui Ding, Sagar Vaze, Francois Porcher, Feng Li, Siyuan Li, Aishwarya Kamath, Ho Kei Cheng, Piotr Dollár, Nikhila Ravi, Kate Saenko, Pengchuan Zhang, and Christoph Feichtenhofer. Sam 3: Segment anything with concepts, 2025. URL <https://arxiv.org/abs/2511.16719>.

- [48] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion, 2024. URL <https://arxiv.org/abs/2303.04137>.
- [49] Xiaohua Zhai, Basil Mustafa, Alexander Kolesnikov, and Lucas Beyer. Sigmoid loss for language image pre-training, 2023. URL <https://arxiv.org/abs/2303.15343>.
- [50] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.